SECURITY PROJECT · DETECTION ENGINEERING

Building a Purple-Team Detection Lab with Wazuh

From a bare SIEM install to a working attack → detect → alert loop, mapped to MITRE ATT&CK.

Md Aman Bhuiyan · Security Engineer, Vivasoft Limited · Red Team & Bug Bounty background
Platform: Wazuh 4.14 on an isolated VMware lab

*All screenshots in this document are from the author's own isolated lab. Private IP addresses, browser URL/bookmark bars, usernames, and bot identifiers have been **redacted** (blacked out) for publication.*

1 Objectives

I set out to go beyond "install a SIEM and look at the dashboard" and instead practice the actual craft of a **detection engineer**, using my offensive background as the engine:

- Stand up the open-source **Wazuh** XDR/SIEM platform and connect multiple endpoints.
- Run a **purple-team loop**: launch an attack, observe what the SIEM detects (and misses), then **write custom detection rules** to close the gaps.
- Map every detection to **MITRE ATT&CK**, understand the compliance context Wazuh adds, and wire up **real-time alerting**.
- Produce something portfolio-worthy: documented, reproducible, and shareable.

2 Architecture

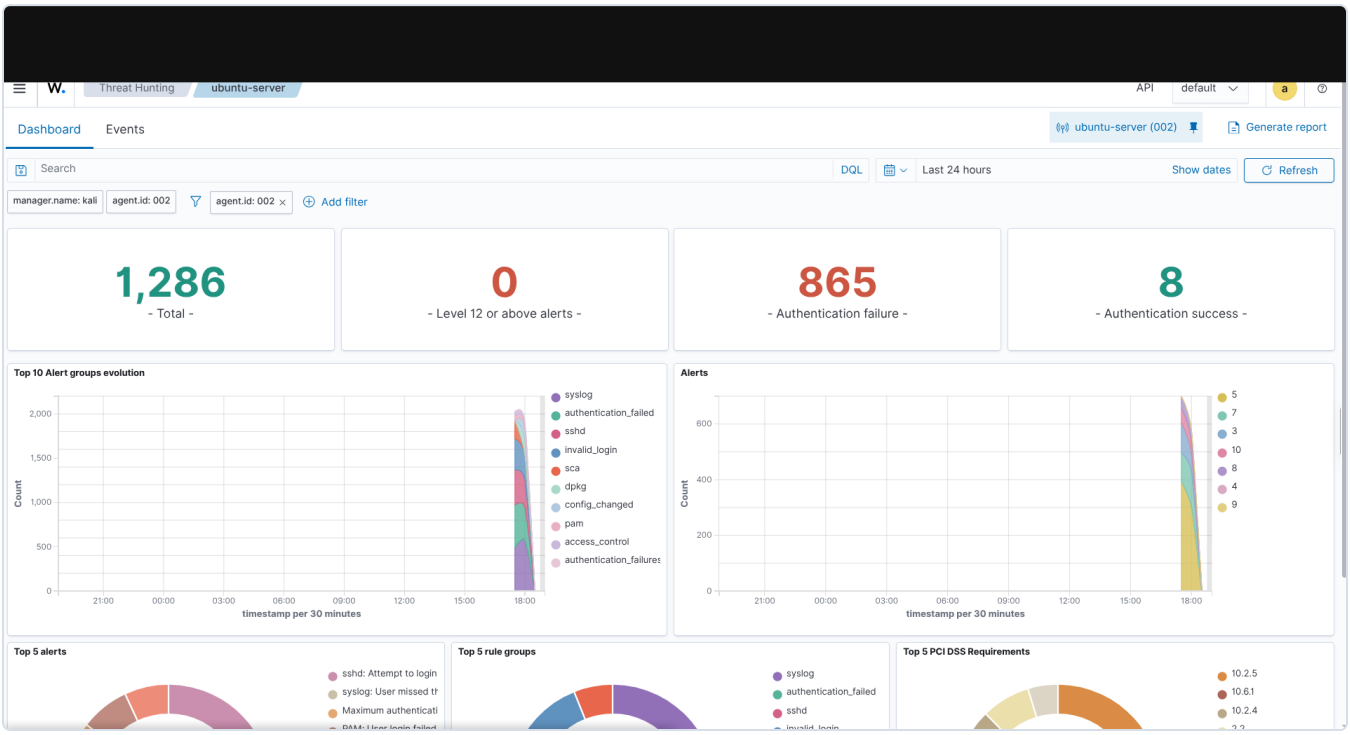
The whole lab runs in VMware on an isolated network so the manager and agents can reach each other while staying off the real network.

- **Kali Linux** — Wazuh **manager**, **indexer**, and **dashboard** (consolidated for the lab), plus attacker tooling (`hydra`, `curl`).
- **Ubuntu Server** — Wazuh **agent**, plus **Apache + MariaDB + PHP + DVWA** as the vulnerable web target.
- **Windows** — Wazuh **agent** for endpoint telemetry.

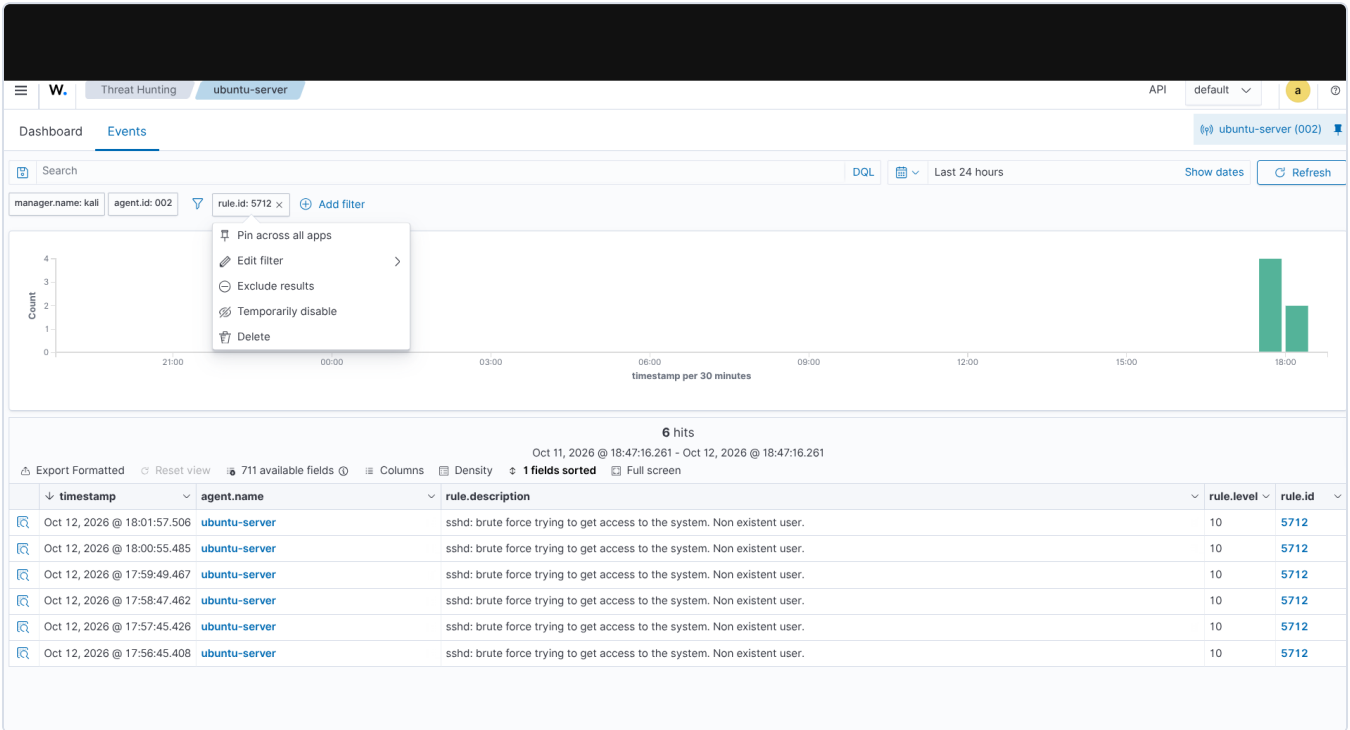
> **Core mental model:** *the agent collects, the manager decides.* Agents ship raw logs; the manager runs the decoders and rules that produce alerts — and the integrations that forward them. That single idea explains most of how Wazuh behaves — and caused one of my most instructive mistakes (Section 4).

3 What I built, step by step

With a Linux agent connected and reporting *Active*, I started an SSH brute force from Kali (`hydra`). Wazuh caught it out of the box.

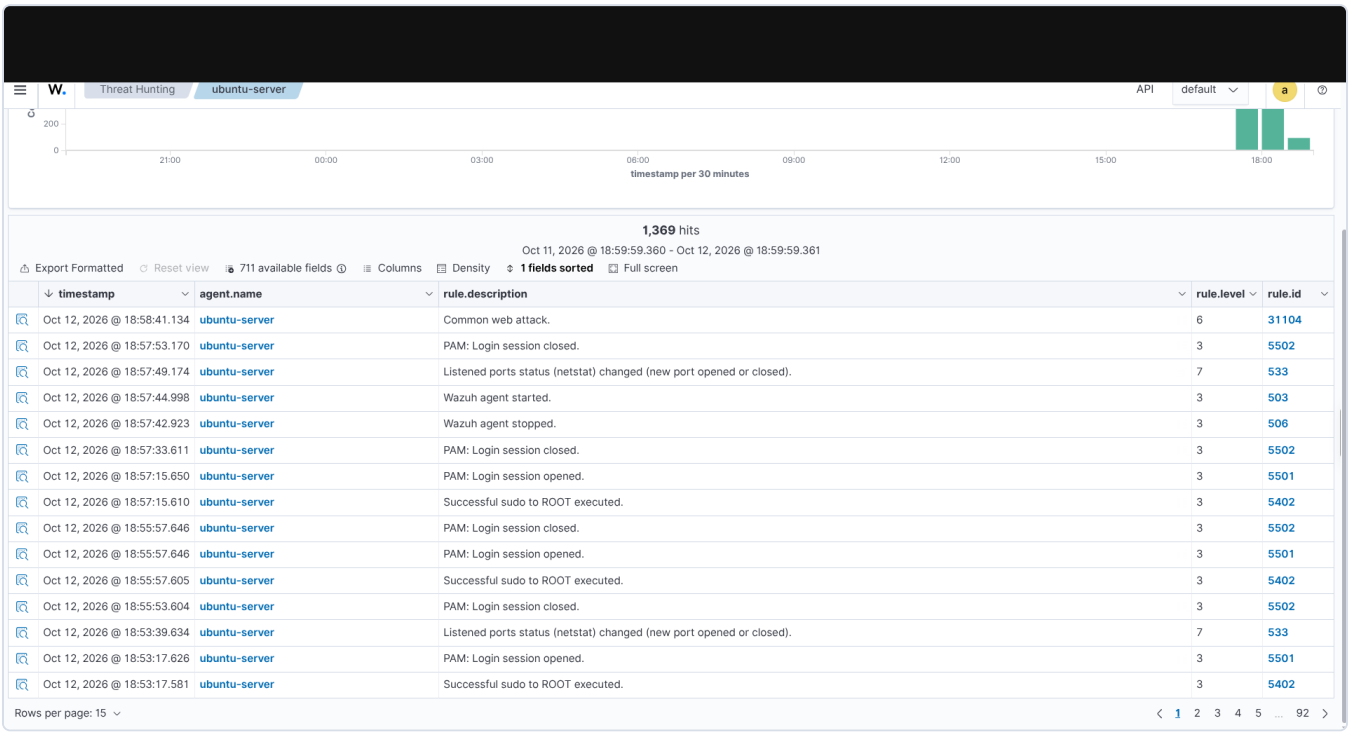


Dashboard overview after the brute-force run. 865 authentication failures against just 8 successes — the lopsided ratio that *is* the signature of a brute force. Wazuh also auto-maps the activity to PCI DSS requirements (lower-right widget).

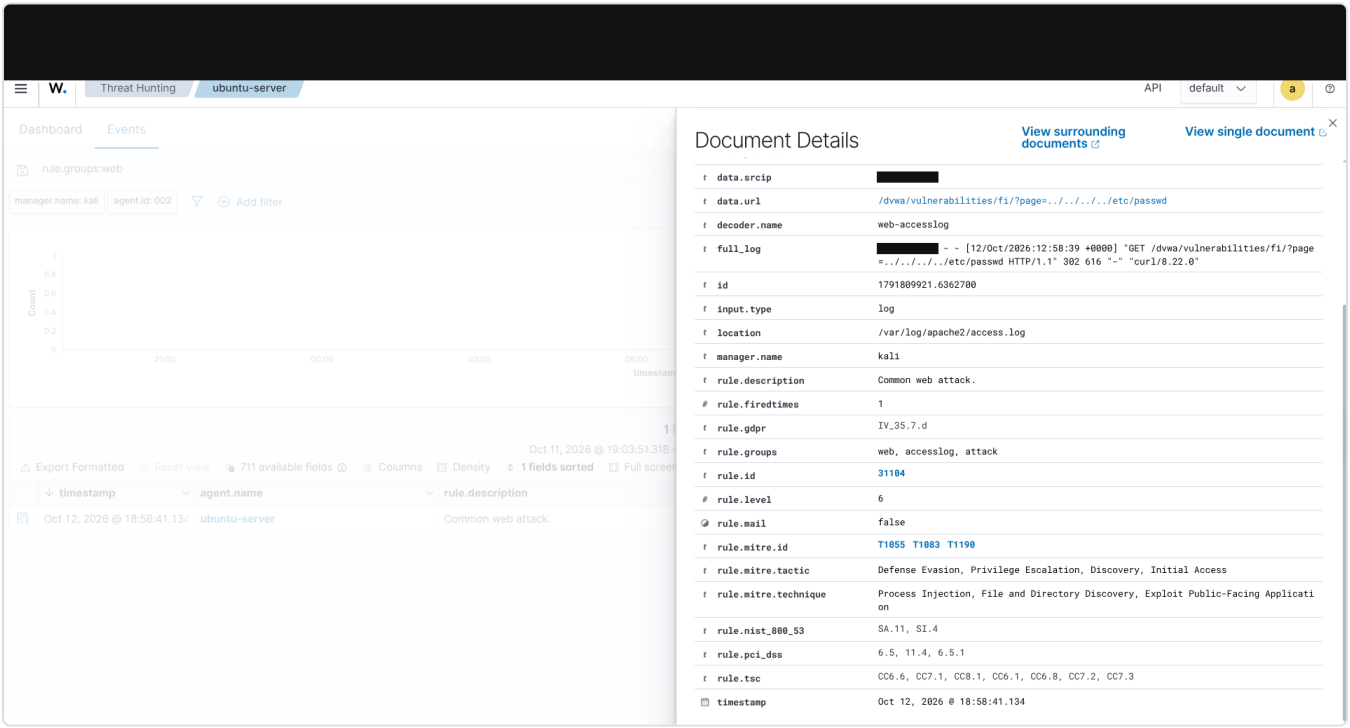


Events filtered to `rule.id:5712`. Individual failed logins are correlated into a single high-severity "SSHD brute force" verdict at level 10, mapped to ATT&CK **T1110**. Correlation — turning noise into one meaningful alert — is the point of a SIEM.

Next I added a completely different attack class: I stood up **DVWA** and enabled Apache access-log collection on the agent (Wazuh does **not** watch the web log by default), then fired SQL-injection and path-traversal payloads.



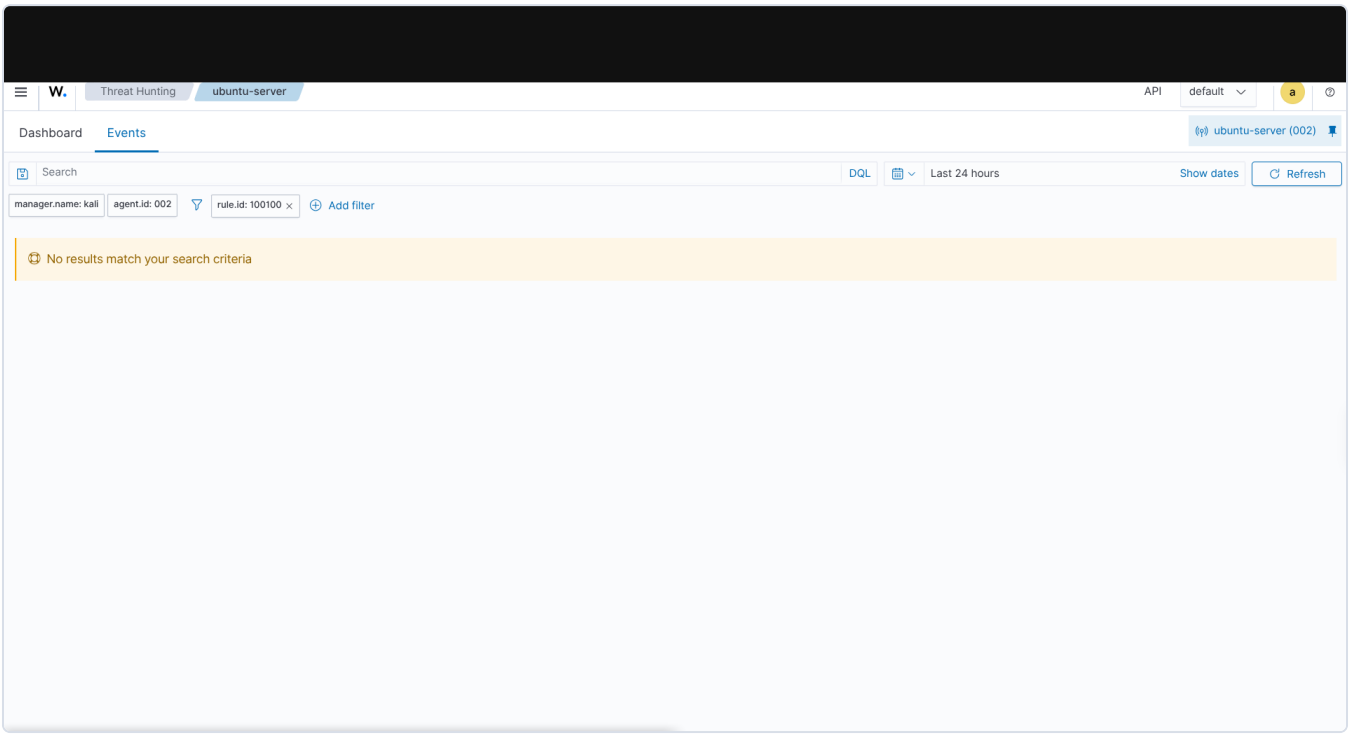
A real monitored host. The web attack fires as rule **31104** "Common web attack" (top row), surrounded by ordinary host activity — sudo-to-root, PAM sessions, and listening-port changes from installing the web stack.



Anatomy of a detection. The `web-accesslog` decoder parsed the raw request into fields; the event is auto-tagged to ATT&CK (**T1055 / T1083 / T1190**) and to compliance frameworks (PCI DSS, NIST). **Source IP redacted.**

4 Challenges & how I solved them

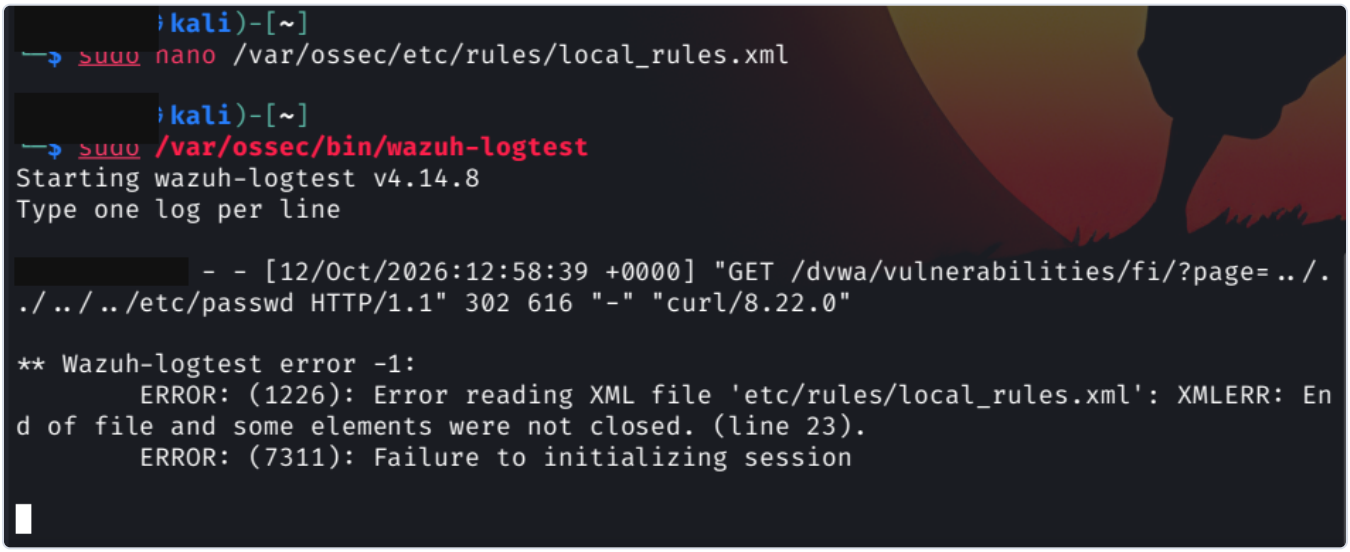
The built-in web alert for a `/etc/passwd` traversal was only a generic level-6 note, so I wrote a custom rule to escalate it. The first attempt returned nothing — and the reason was the most valuable lesson of the project.



The instructive failure. Filtering for the brand-new rule (`rule.id:100100`) returns "No results." The rule logic was fine — the rule was simply in the wrong place.

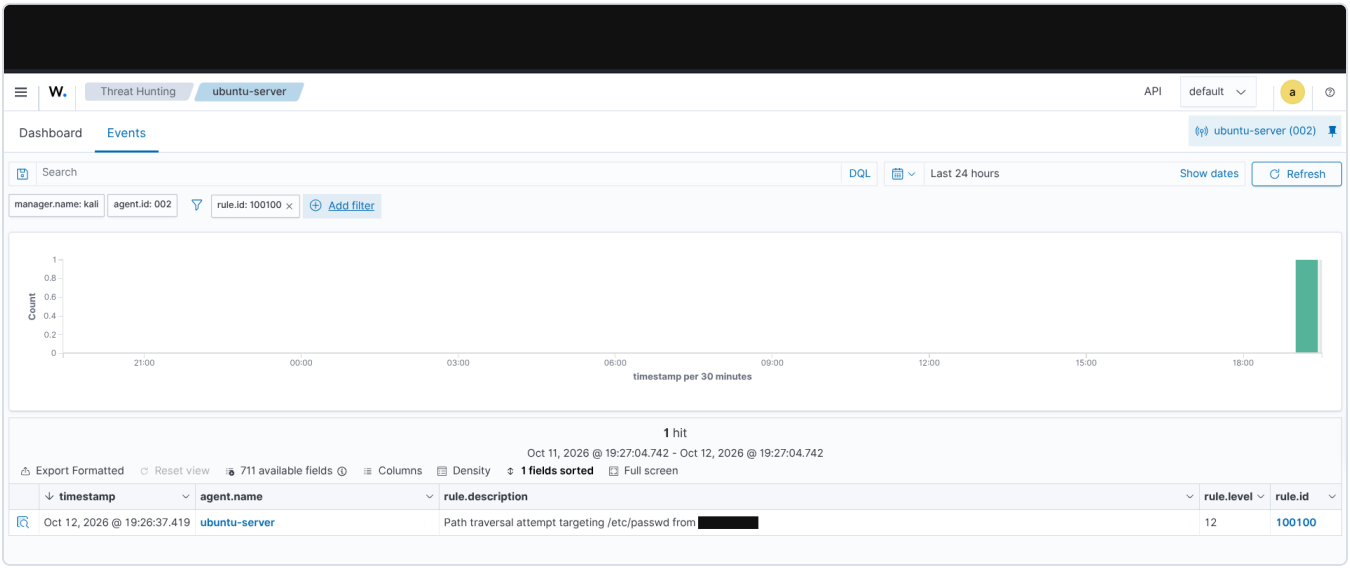


Root cause. The rule file was created on the *agent* (host prompt "victim"), but rules are only evaluated on the **manager**. The agent collects; the manager decides. **Username redacted.**



Validate before you ship. Running `wazuh-logtest` on the manager caught a malformed-XML error *before* any restart — precisely why you validate first, so a syntax error can never take the running SIEM offline. **IP in the sample log redacted.**

Challenge	Root cause	Fix
Agent couldn't reach the manager ("No route to host")	Manager and agent on **different VMware network modes / subnets**	Put both VMs on the same NAT segment; confirmed with `ping` first
Couldn't copy/paste into the Ubuntu **Server** console	Headless server has no GUI clipboard integration	Enabled SSH; worked from the Kali terminal where paste is native
Custom rule produced **no results**	Rule written **on the agent**; rules run on the **manager**	Moved `local_rules.xml` to the manager's `/var/ossec/etc/rules/`
Manager threw `XMLERR: elements were not closed`	Malformed / duplicated XML tags	Rebuilt with balanced tags; re-validated with `wazuh-logtest`
Rule still didn't appear after the fix	Manager loads rules only at startup; old events don't re-match	Restarted the manager, then fired a **fresh** attack

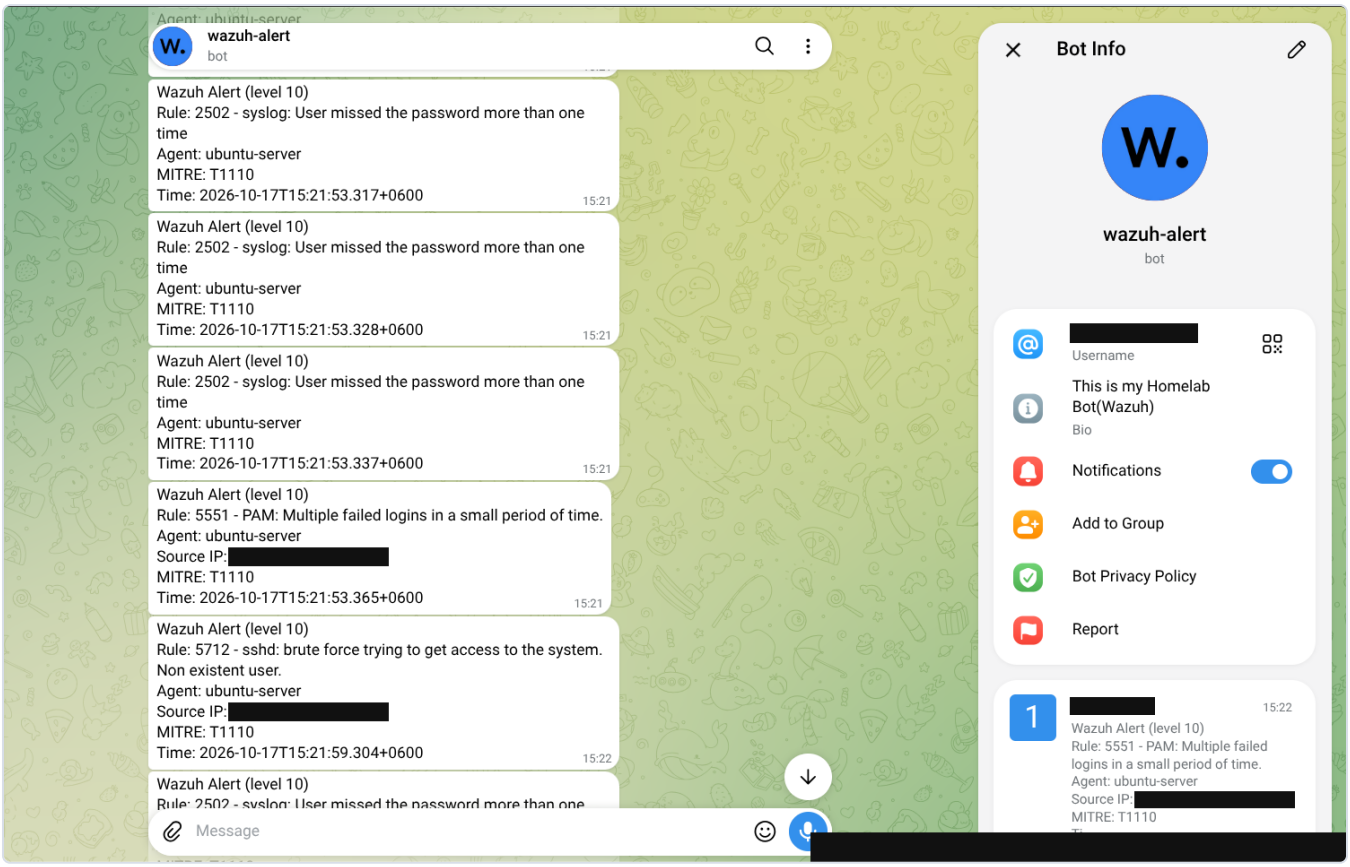


The payoff. After moving the rule to the manager, fixing the XML, restarting, and firing a fresh attack, custom rule **100100** fires at **level 12** — a generic level-6 alert upgraded into a specific, high-severity, attacker-attributed detection. **IP redacted.**

5 Real-time Telegram alerting

To make the lab feel operational, I forwarded high-severity detections to a Telegram bot in real time through Wazuh's `wazuh-integratord` daemon. A custom script on the **manager** parses each alert and posts it to the Telegram API; a `level 10` threshold in the `<integration>` block keeps routine noise off the phone while still delivering the SSH brute force (rule 5712) and my custom rule (100100).

Getting it working was itself a string of ordinary, instructive failures: the bundled Python lived at a different path than the wrapper assumed; pasted here-documents kept writing shell commands into the script; editor indentation errors crept in; and — the classic — I first tried to use my *phone number* as the Telegram chat ID instead of the numeric ID from @userinfobot. Each was a quick fix once the traceback was read carefully.



Alerts on the phone. During a live `hydra` brute force against the Ubuntu agent, correlated detections — rules **5712** (sshd brute force), **5551** (PAM multiple failed logins) and **2502** (repeated password failures), all level 10 and tagged **T1110** — arrive in Telegram within seconds. The `level 10` threshold keeps routine noise off the phone. **Source IP and bot username redacted.**

6 Key takeaways

- **Detection engineering is a loop, not a setting.** Attack → observe → read the raw event → write/tune a rule → validate → verify. Repeating that loop is the skill.
- **Telemetry quality decides everything.** Wazuh caught SSH brute force out of the box but saw nothing on the web until I explicitly added the Apache log. You can't detect what you don't collect.
- **Read the error, don't guess.** Nearly every blocker in this project — networking, rule placement, XML, the Telegram setup — was solved by reading the last line of the message carefully.
- **Specific beats loud.** Going from a generic level-6 alert to a named level-12 detection with the attacker's IP is the difference between noise and an actionable signal.
- **An offensive background is an asset on defense.** Knowing how the attack really works made the detection obvious to design.

7 Next steps

- More custom Linux rules: web-shell upload, directory scanning (repeated 404s), reverse-shell process spawn.
- Install **Sysmon** on the Windows agent and author a Windows detection (e.g. suspicious PowerShell download).
- Build a **MITRE ATT&CK Navigator** heatmap of covered techniques (T1110, T1190, T1083, …).
- Add **active response** to auto-block an attacker IP on a high-severity rule.

- Route alerts by severity (e.g. level 12+ to a separate priority channel).
- Extend to an **Active Directory** scenario for AD-specific attack detection.

Author: Md Aman Bhuiyan — Security Engineer, Vivasoft Limited. This project uses intentionally vulnerable software (DVWA) and was run entirely on an isolated virtual network against machines the author owns. Screenshots redacted to remove private IPs, URLs, usernames, and bot identifiers.